

1
Week1
Day5
TopicsES6+
Focus

Topic 1 — let, const vs var: Scope & Hoisting

var function-scoped hota hai aur hoist hota hai — matlab declaration code execute hone se pehle top pe move ho jaati hai. let aur const block-scoped hain (curly braces {} ke andar hi valid). const ko declare karte waqt initialize karna zaroori hai aur reassign nahi kar sakte, lekin agar const object hai to uske andar values change ho sakti hain.

var — Avoid

Function scope, hoisted, re-declarable. Bugs ka source.

let — Use karo

Block scope, not hoisted (TDZ), reassignable. Best for loops.

const — Prefer

Block scope, must initialize. Default choice hona chahiye.

```
// var — hoisting example (avoid!)
console.log(x); // undefined — hoisted!
var x = 5;

// let — block scoped
for (let i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}
// console.log(i) — ReferenceError! i only inside loop

// const — immutable binding
const PI = 3.14;
// PI = 3; — TypeError: Assignment to constant variable

const user = { name: 'Rahul' };
user.name = 'Priya'; // OK! Object mutation is allowed
```

Topic 2 — Arrow Functions & Implicit Return

Arrow functions (=>) ek shorter syntax provide karti hain. In■■■ apna 'this' nahi hota — ye lexical this use karti hain (surrounding context ka this). Single-expression functions mein return keyword optional hai (implicit return). Single parameter hone par () bhi optional hai.

```
// Regular function
function add(a, b) { return a + b; }

// Arrow function — same output, concise syntax
const add = (a, b) => a + b;

// Single param — no () needed
```

```

const double = n => n * 2;

// Multi-line arrow function
const greet = (name) => {
  const msg = `Hello, ${name}!`;
  return msg;
};

// Return object – wrap in parentheses!
const getUser = (name) => ({ name, role: 'admin' });

// 'this' – arrow vs regular
function Timer() {
  this.count = 0;
  setInterval(() => {
    this.count++; // 'this' = Timer object (lexical)
  }, 1000);
}

```

Topic 3 — Template Literals & Tagged Templates

Template literals backtick (`) use karte hain. `${expression}` se variables aur expressions embed kar sakte hain. Multi-line strings bhi natively supported hain — no `\n` needed. Tagged templates advanced feature hai jahan function string process karta hai.

```

// Old ES5 way – messy concatenation
var msg = 'Hello, ' + name + '! Age: ' + age;

// ES6 Template literal – clean!
const msg = `Hello, ${name}! Age: ${age}`;

// Expression inside ${}
const price = `Total: Rs.${(100 * 1.18).toFixed(2)}`;

// Multi-line string
const html = `
<div>
<h1>${name}</h1>
</div>
`;

// Tagged template (advanced)
function highlight(strings, ...values) {
  return strings.reduce((result, str, i) =>
    result + str + (values[i] ? `<b>${values[i]}</b>` : ''), ''
  );
}

const output = highlight`Name: ${name}, Age: ${age}`;

```

Topic 4 — Destructuring: Arrays & Objects

Destructuring se arrays ya objects se values directly variables mein extract kar sakte hain. Variables rename kar sakte hain, default values set kar sakte hain, aur nested structures bhi destructure ho sakti hain. Function

parameters mein bhi use hota hai.

```
// Array Destructuring
const colors = ['red', 'green', 'blue'];
const [first, second, third] = colors;

// Skip elements
const [, , last] = colors; // last = 'blue'

// Swap – no temp variable!
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1

// Object Destructuring
const student = { name: 'Priya', age: 21, city: 'Delhi' };
const { name, age } = student;

// Rename while destructuring
const { name: studentName, city: hometown } = student;

// Default values
const { grade = 'A', name: nm } = student;

// Nested destructuring
const { address: { pincode } } = { address: { pincode: 110001 } };

// Function param destructuring
const greet = ({ name, age }) => `${name} is ${age} years old`;
```

Topic 5 — Default Parameters & Rest/Spread Operator

Default parameters allow karte hain ki agar argument pass na ho to ek default value use ho. Spread (...) array/object ko expand karta hai. Rest (...) multiple arguments ko ek array mein collect karta hai. Dono ka syntax same hai — position se decide hota hai kaunsa hai.

Default Params

Function mein argument na mile to fallback value use hoti hai.

Spread (...)

Array/object expand karta hai — copy, merge, function call mein.

Rest (...)

Remaining arguments ya properties ko array mein collect karta hai.

```
// Default parameters
const greet = (name = 'Guest', role = 'user') =>
  `Hello ${name}, you are a ${role}`;
greet(); // 'Hello Guest, you are a user'
greet('Rahul'); // 'Hello Rahul, you are a user'

// Spread – expand arrays
const arr1 = [1, 2, 3];
const arr2 = [4, 5];
const combined = [...arr1, ...arr2]; // [1,2,3,4,5]

// Clone & merge objects
const base = { theme: 'dark', lang: 'en' };
const settings = { ...base, lang: 'hi' }; // lang overrides

// Spread in function call
const nums = [10, 20, 5];
```

```
Math.max(...nums); // 20

// Rest – collect unlimited args
const sum = (...numbers) =>
  numbers.reduce((acc, n) => acc + n, 0);
sum(1, 2, 3, 4); // 10

// Rest in destructuring
const [head, ...tail] = [1, 2, 3, 4];
// head = 1, tail = [2, 3, 4]
const { a, ...rest } = { a: 1, b: 2, c: 3 };
// a = 1, rest = { b: 2, c: 3 }
```

Learning Outcome

Is session ke baad tum confidently modern ES6+ JavaScript likh paoge.

Tumhe var vs let vs const ka scope samajh aayega, arrow functions likhna aayega, template literals se dynamic strings banana aayega, destructuring se clean code likhna aayega, aur spread/rest operator effectively use karna aayega.